Fonoa

GoTalks 2026.01

Fonoa

Preventing Drift by Design

Code generation, contracts, and Go tooling

Who am I?

Tomas Espeleta
Senior Software Engineer

tomas.espeleta@fonoa.com

Agenda

01 Code generation and SSOT

02 Generating models and validators

03 Lisa DSL

04 Conclusions

05 Appendix: more examples

Fonoa

Distributed Systems

....means **Distributed Truth**

Facts about the system are fragmented.

At Fonoa we have multiple:

- Schemas
- Validations
- Data Models
- Documentation
- Model Mappings

Code

```
type MyRequest struct {
  CountryCode string `json:"country_code"`
}
```

Documentation

country_code: ISO 3166-1 alpha-2 uppercase country code the transaction is taking place in.

Why the fragmentation?

*We need optimisations that solve **specific problems***

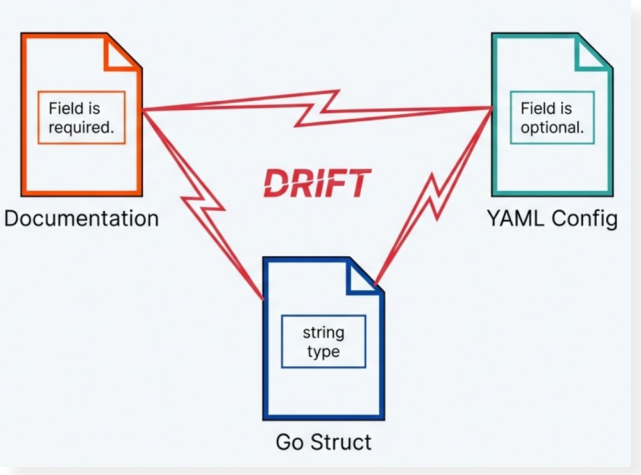
- **Contracts** define a **shared reality**
  - Need to be optimised for **reading** and **change control**
- **Data Models**: we need them **language-specific!**
- **Data Validations**: we want them **fast!**
- **Documentation**: we want it beautiful and **readable**!
- **Model Mappings**: we want them… to disappear! :(

Each specific problem needs a specialised technology

The problem: Drift

Drift is the gap between what we think the system does and what it actually does

- Documentation go stale
- Comments lie
- APIs diverge
- Different models
- Different life cycles



We end up spending a lot of time synchronizing them!

Do not repeat yourself?

DRY...

Changes are **expensive** and error-prone

Change becomes a maintenance nightmare.

...or not to DRY?

DRY **increases coupling**

Difficult with multiple technologies

Repetition is generally preferable to coupling

"A little copying is better than a little dependency"
- Rob Pike

Single Source Of Truth

And its challenges

Authoritative representation in **one** place

The Challenge
Choosing an expressive and easy to read SSOT is important for maintenance

Targets
Systems need specialized artifacts (**Go code** for speed, **JSON** for UI, **HTML** for browsers)

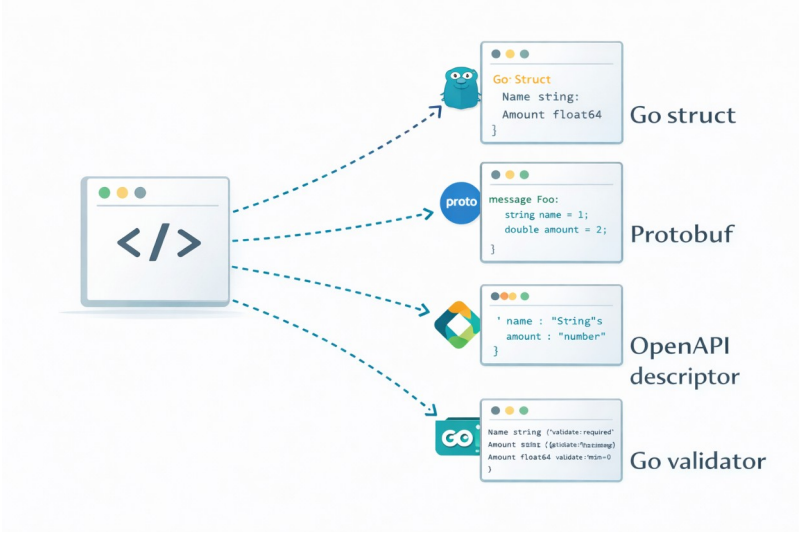
The Solution
Code generation: “copy and paste **at scale**!!!”

Code Generation

A better solution

- Code generation **solves coupling** by **embracing repetition**
- **Automates** the synchronization of multiple “copies”
- Multiple targets (e.g. documentation and multiple programming languages)

SSOT



Targets

First example:

Models and API validators

API Model

Declarative API model, ***generated go structs***

SSOT:
Custom YAML

```
TransactionRequest:
 properties:
   language_code:
     type: string
     description: "Language of the transaction document represented as ISO 639-1 uppercase language code. Required when document generation is enabled. Defaults
to EN (English) if not provided."
     example: "EN"
   country_code:
     type: string
     description: "ISO 3166-1 alpha-2 uppercase country code the transaction is taking place in."
     example: "PT"
different when operation regime is EXPORT."
   transaction_date:
     type: google.protobuf.Timestamp
     description: "RFC3339 datetime string representing the point in time when transaction is executed."
     example: "2021-01-07T01:00:00+00:00"
   items:
     type: repeated TransactionLineItem
     description: "Goods, services, and discounts exchanged in a transaction."
   supplier:
     type: TransactionEntity
   total_amount:
     type: common.Decimal
     description: "A high precision decimal representing the total amount. If you are using the legacy integer representation for this field refer to this
documentation on how to migrate to decimals: https://docs.fonoa.com/reference/amounts-and-rounding"
     example: 122.00
   issue_date:
     type: google.protobuf.Timestamp
     description: "The date when the transaction was issued by an external application."
```

Fonoa

API Model (I)

The tool: go template

Tool:
go template

```
package {{ .PackageName }}

{{- if .Imports}}
import (
{{- range .Imports}}
    {{.}}
{{ end}}
){{- end }}

{{- range .Types }}
{{- $modelName := .Name }}

{{ (printf "%s is in need of a description." .Name) | commentBlock }}
{{- if .OneOfTypes }}
type {{.Name}} struct {
    {{- range .OneOfTypes }}
        {{ .Name }}
    {{- end}}
}

...
```

Fonoa

API Model (III)

The output: Generated go srtruct for our external API

Output:
go validator

```go
type TransactionRequest struct {
   // CountryCode: ISO 3166-1 alpha-2 uppercase country code the transaction is taking place in.
   CountryCode string `json:"country_code,omitempty"`

   // IssueDate: The date when the transaction was issued by an external application.
   IssueDate *string `json:"issue_date,omitempty"`

   // Items: Goods, services, and discounts exchanged in a transaction.
   Items []*TransactionLineItem `json:"items"`

   // TotalAmount: A high precision decimal representing the total amount. If you are using the legacy integer representation for
this field refer to this documentation on how to migrate to decimals: https://docs.fonoa.com/reference/amounts-and-rounding
   TotalAmount *json.Number `json:"total_amount,omitempty"`

   // IssueDate: The date when the transaction was issued by an external application.
   IssueDate *string `json:"issue_date,omitempty"`

   // TransactionDate: RFC3339 datetime string representing the point in time when transaction is executed.
   TransactionDate *string `json:"transaction_date"`

   // TransactionId: Unique external transaction identifier provided by the customer.
   TransactionId string `json:"transaction_id,omitempty"`

   // [...]
}
```

Fonoa

API Validators

Declarative API Validation, ***generated go validators***

SSOT:
Custom YAML

```yaml
ExportRequest:
 properties:
   format:
     type: string
     description: "One of the supported formats: saft, csv-pt, csv-hr, internal-act-hr, csv-generic, csv-tr"
     validation:
     - required: true
     - value_in: ['saft', 'csv-pt', 'csv-generic', 'csv-tr']
   date_from:
     type: google.protobuf.Timestamp
     description: "Starting date to filter transactions"
     validation:
     - rfc3339_string: true
     - required: true
       when: m.Format != "csv-tr"
     - must: isNullOrEmpty(m.DateTo)
       when: m.Format == "csv-tr"
       with_message: "{{.propertyName}} must be empty for the 'csv-tr' format"
```

Fonoa

API Validators (I)

The tool: go template

Tool:
go template

```
func (r rule{{$modelName}}) ValidateStruct(ctx context.Context, m external.{{.Name}}) error {
{{- if .Fields }}
   ctx = helpers.AddAsParent(ctx, m)
   structErrors := validation.ValidateStructWithContext(ctx, &m,
   {{- range .Fields}}
   {{- $elementType := (printf "%s%s" (upper $country) .ElementType) -}}
   {{- if .Validation }}
       validation.Field(&m.{{ toPascalCase .Name}}, {{ validationsList .Validation $country .Name -}}
               {{- if and .CanBeNested (.HasValidations $allTypes) (not .IsCollection) -}}
                   , NewRule{{$elementType}}(ctx, "{{.Name}}")
               {{- end }}),
   {{- else if and .CanBeNested (.HasValidations $allTypes ) (not .IsCollection) }}
       validation.Field(&m.{{ toPascalCase .Name}}, NewRule{{$elementType}}(ctx, "{{.Name}}")),
   {{- end}}
   {{- end}}
   )
   {{- range .Fields}}
   {{- if and .IsCollection (.HasValidations $allTypes) }}
       {{- $elementType := (printf "%s%s" (upper $country) .ElementType) }}
       structErrors = helpers.ValidateNestedCollection(ctx, "{{.Name}}", m.{{ toPascalCase .Name}}, structErrors,
NewRule{{$elementType}}(ctx, "{{.Name}}"))
   {{- end}}
   {{- end}}
   return helpers.FlattenErrors(structErrors)
{{- end }}

[...]
```

Fonoa

API Validators (II)

The tool: astutils as refinement of the output.

Tool: go/astutil

```go
func toGo(expression string, country string) string {
   parsed, err := parser.ParseExpr(expression)
   if err != nil {
      panic("error parsing go expression: '" + expression + "'")
   }
   result := astutil.Apply(parsed, func(cr *astutil.Cursor) bool {
      n := cr.Node()

      // some AST replacements to implement shorter syntax for certain functions
      if call, ok := n.(*ast.CallExpr); ok {
         if call.Fun == nil {
            return true
         }
         if id, isId := call.Fun.(*ast.Ident); isId {
            if id.Name == "pathValue" && len(call.Args) > 0 {
               cr.Replace(&ast.CallExpr{
                  Fun: ast.NewIdent("helpers.PathValue"),
                  Args: append([]ast.Expr{ast.NewIdent("ctx")},
                     &ast.IndexExpr{
                        X:     ast.NewIdent("expMap" + strings.ToUpper(country)),
                        Index: call.Args[0],
                     }),
               })
            }
            if funcName, exists := functions[id.Name]; exists {
               cr.Replace(&ast.CallExpr{
                  Fun:  ast.NewIdent(funcName),
                  Args: call.Args,
               })
            }
         }
      }

      if ident, ok := n.(*ast.Ident); ok {
         switch ident.Name {
         case "null":
            // null -> nil
            cr.Replace(ast.NewIdent("nil"))
         }
      }
```

API Validators (III)

The output: Generated go validator for our external API

Output: go validator

```
func NewRuleReportedTransactionRequest(ctx context.Context, fieldName string) ruleReportedTransactionRequest {
   return ruleReportedTransactionRequest{fieldCtx: helpers.AddFieldToPath(ctx, fieldName)}
}

func (r ruleReportedTransactionRequest) ValidateStruct(ctx context.Context, m external.ReportedTransactionRequest) error {
   ctx = helpers.AddAsParent(ctx, m)
   structErrors := validation.ValidateStructWithContext(ctx, &m,
      validation.Field(&m.CountryCode, rules.Required, NewCountryCodeValidator()),
      validation.Field(&m.TaxAuthorityId1, rules.MaximumLength(150)),
      validation.Field(&m.TransactionDate, rules.Required, rules.RFC3339String),
      validation.Field(&m.TransactionId, rules.MaximumLength(150), rules.Required),
      validation.Field(&m.TransactionNumber, rules.MaximumLength(50)),
   )
   return helpers.FlattenErrors(structErrors)
}
[...]
res.Amount = mapping.ToExternalMoneyAsJsonNumber(ctx, in.Amount, helpers.PathValue(ctx, expMap[".CurrencyCode"]))
```

- We experimented with HTML generation with another template.
- We also **generate OpenAPI** from the custom descriptors (`github.com/getkin/kin-openapi/openapi3`)

Fonoa

Internal Models

A brief recap...

- We have a centralised repository of **protobuf** models
- We generate **go** specific structs with buf tool, we try to use them everywhere

External model

```
type TransactionRequest struct {
   CountryCode     string                 `json:"country_code,omitempty"`
   IssueDate       *string                `json:"issue_date,omitempty"`
   Items           []*TransactionLineItem `json:"items"`
   TotalAmount     *json.Number           `json:"total_amount,omitempty"`
   IssueDate       *string                `json:"issue_date,omitempty"`
   TransactionDate *string                `json:"transaction_date"`
   TransactionId   string                 `json:"transaction_id,omitempty"`
}
```

Internal model

```
type Transaction struct {
  state         protoimpl.MessageState
  sizeCache     protoimpl.SizeCache
  unknownFields protoimpl.UnknownFields

  CountryCode      string                 `protobuf:"..."`
  InvoiceIssueDate *timestamppb.Timestamp `protobuf:"..."`
  Items            []*TransactionItem     `protobuf:"..."`
  TotalAmount      *Money                 `protobuf:"..."`
  TransactionDate  *timestamppb.Timestamp `protobuf:"..."`
  TransactionUid   string                 `protobuf:"..."`
}
```

Fonoa

Model Mappers

- Fields mapped **automatically** by name and structure
- All the exceptions can be specified in a custom descriptor

```yaml
- internal: total_net_amount
  expression: ToInternalMoneyFromJsonNumberRef(ctx, in.TotalNetAmount, in.CurrencyCode)
```

SSOT:
Custom YAML

```
res.ActivityCode = in.ActivityCode
res.CountryCode = in.CountryCode
res.TotalNetAmount = mapping.ToInternalMoneyFromJsonNumberRef(ctx, in.TotalNetAmount,
in.CurrencyCode)
res.CurrencyCode = helpers.ToUpper(in.CurrencyCode)
res.Direction = m.ToInternalTransactionDirection(ctx, in.Direction)
res.InvoiceIssueDate = utils.StringRefToPbTimestampFlexible(in.IssueDate)
res.Issuer = m.ToInternalTransactionNonOnboardedEntity(ctx, in.Issuer)
res.Items = m.ToInternalTransactionLineItems(ctx, in.Items)
res.LanguageCode = in.LanguageCode
res.Note = in.Note
```

Output:
go validator

Lifecycle

Go helps us to keep control!

- **In the CI, on each git push:**
  - **We generate everything:** models, validators, mappers (FAST!)
  - **We run all the tests:** no merge without passing tests
- Breaking changes are flagged by:
  - Incompatible types in assignments (!)
  - Failing contract tests
- More difficult to find: **missing mappings**
  - Default mapping by field name mitigates this

Why not interpreting?

01

Easy debugging

Compiled Go code is much easier to debug than interpreted code.

02

Type Safety

Compiled Go code catches errors that a YAML parser won't.

03

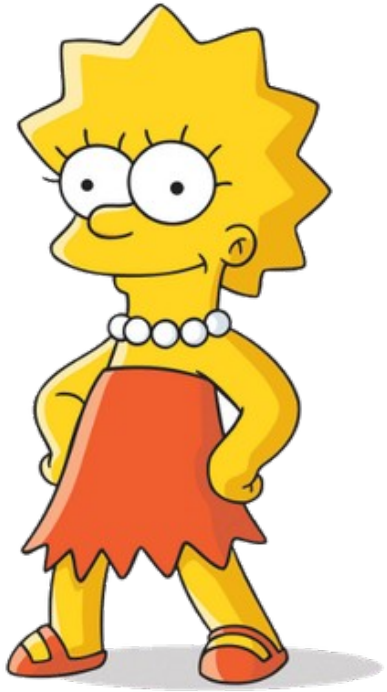
Performance

Evaluating expressions at runtime can be slow.

Although, this is a micro optimization!

A more complex use case

Lisa

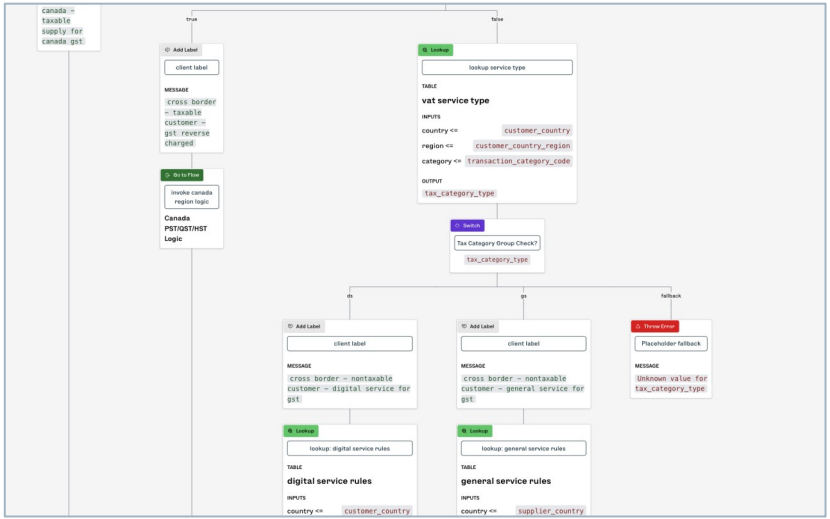


Tax logic rules

Slightly more than a rules engine

In the tax determination engine:

- We let **non-technical users** to edit tax logic with a no-code approach
- The backend of the **editor** uses an in-memory hierarchical model (**JSON)**



Main Challenge: what should we store?

JSON is not great for reading and change management

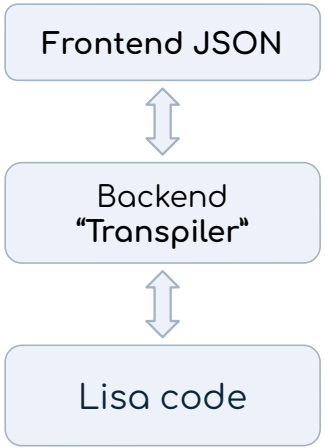
We store “Lisa”

A go-derived DSL

We decided to define a **subset of go** for the logic definition, that we call “Lisa”.

Lisa supports:

- Assignments
- Expressions
- Conditionals
- Loops (very limited)
- Global variables



Volatile

SSOT

Lisa example

Lisa is a simple go subset

SSOT: lisa

```
package logic

// Apply Car Rental Tax
func applyCarRentalTax() {
   // Is there an airport code for the pickup?
   if transit_origin_airport_code != "" {
      // add airport fee to breakdown
      addToBreakdown(transit_origin_country, transit_origin_airport_code, "carRental", "std")
   } else {
      // Is there an airport code for the drop off?
      if transit_destination_airport_code != "" {
         // add airport fee to breakdown
         addToBreakdown(transit_destination_country, transit_destination_airport_code, "carRental",
"std")
      }
   }
   // lookup all the regions mapped to a zip code
   transaction_country_regions = lookup("zip_to_regions", country_regions, Values{"zip": zip_code})
   ...
```

Fonoa

Frontend json

A target for rendering

Output:
Lisa UI json


```
{
 "flows": [
   {
     "id": "applyCarRentalTax",
     "name": "Apply Car Rental Tax",
     "block": {
       "id": "applyCarRentalTax",
       "endsWith": "return",
       "statements": [
         {
           "id": "4",
           "label": "Is there an airport code for the pickup?",
           "type": "decision",
           "decideWith": "condition",
           "condition": {
             "predicate": {
               "left": {
                 "kind": "parameter",
                 "value": "transit_origin_airport_code"
               },
               "operator": "!=",
               "right": {
                 "kind": "literal",
                 "value": ""
               }
             }
           },
```

Lisa to Frontend json

We parse lisa into our json UI format

Tool:
ast package

```
func (t *LisaTranspiler) toOperand(expr ast.Expr) (model.Operand, error) {
   switch v := expr.(type) {
   case *ast.Ident:
      str := identifier(v)
      if str == "true" || str == "false" {
         return model.Operand{Kind: model.LITERAL_OP, Value: str}, nil
      } else {
         return model.Operand{Kind: model.PARAMETER_OP, Value: str}, nil
      }

   case *ast.BasicLit:
      str, err := literalToString(v)
      if err != nil {
         return model.Operand{}, fmt.Errorf("cannot convert literal: %s", v.Kind)
      }

      return model.Operand{
         Kind:  model.LITERAL_OP,
         Value: str,
      }, nil

   default:
      return model.Operand{}, fmt.Errorf("could not convert operand with type %T", expr)
   }
}
```

Fonoa

Frontend json to Lisa

Simple and basic string Builder ¯_(ツ)_/¯

Tool:
String Builder

```
func (t *LisaTranspiler) toLisaFunctions(flow model.Flow, allFlows []model.FlowDescription, params paramsMap) (string, error) {
   code := strings.Builder{}
   for _, f := range allFlows {
      t.flowsById[f.ID] = &f
   }
   blockCode, err := t.blockToLisa(flow.Block, params.WithLocal(flow.Parameters))
   if err != nil {
      return "", fmt.Errorf("failed to transpile flows: %w", err)
   }
   funcName := flow.ID
   if len(flow.Name) > 0 {
      code.WriteString("// ")
      code.WriteString(flow.Name)
      code.WriteRune('\n')
   }
   code.WriteString("func ")
   code.WriteString(funcName)
   code.WriteString("(")
   for i, param := range flow.Parameters {
      code.WriteString(param.Name + " " + ToLisaType(param.Type))
      if i < (len(flow.Parameters) - 1) {
         code.WriteString(", ")
      }
   }
   code.WriteString(") {\n")
   code.WriteString(blockCode)
   code.WriteString("}\n")

   return code.String(), nil
}
```

Fonoa

We love go!

Business logic and tooling

go code

Normal, plain, handwritten go code

- No duplication: standard practices

go tools

- go templates
- go parser
- AST manipulation

Source / Target

go as target

Generation **to go**:

- API validators
- XSD validators
- Lisa generation

go as SSOT

Generation **from go:**

- Lisa UI generation
- HTML generation

Summary

Source (SSOT)	Tool	Target (Artifact)
Database / Go Structs	Go Templates	Web pages (HTMX)
API Descriptors	Go Templates Astutils package OpenAPI library	Go Models Go Validators OpenAPI
Lisa Editor -> Lisa Code	String Builder	Go Tax Engine
Lisa Code -> Lisa Editor	Go AST Parser	Visual Editor JSON
XSD Files	Go Templates	Go Models Go Validators

Fonoa

Conclusion

Preventing Drift by design

- **Drift** is inevitable when you expect humans to synchronize artifacts.
  - Humans time is too precious!
- **Code generation** is not about removing repetition
  - It’s about **embracing repetition** and **automating** synchronization
  - Stable and deterministic
- Go fits naturally in this role
  - Strong types catch drift early: errors move to compile time
  - Generated code stays plain, debuggable Go

Thank you!